

Drishti v1 — Launch Status Report

Floor-plan PDF → walkable 3D building | Compiled 18 July 2026 (deploy day) | Repo: D:\3D React Viewer

1. What Drishti is

Drishti turns an architect's CAD floor-plan PDF into a 3D building you can orbit around and walk inside, straight in the browser. Two parts: a **FastAPI server** (Python) that reads the PDF's vector drawing layers and builds a GLB 3D model, and a **React viewer** (three.js) that renders it with realistic materials, sun, sky and room-by-room walk-inside beacons. v1 scope: CAD PDFs fully supported; photos/scans answer with a friendly 'beta' message (no heavy AI model needed on the server, so hosting is cheap).

2. Everything built so far (checklist)

2.1 Parsing engine (server)

Item	Status	Details
Wall extraction	✓ Done	Vector line pairing v3 — walls placed at true drawn positions; phantom walls across doorways eliminated; de-blobbed output mask (square corners, walls ~1.4 in thinner).
Text-scale detection	✓ Done	Reads dimension text on the sheet to establish real-world feet-per-point scale.
Doors	✓ Done	Swing-symbol detection + snapping to wall gaps; vector fallback for junction doors when the raster gap-finder fails; door strips cut proper headers.
Windows	✓ Done	Per-rect evaluation + on-wall test + dedup; found via window layer/tags (Plan B, 2-stage tag search).
Stairs	✓ Done	Stair filter v3 chains short rungs anywhere in the strip list; corrugated stair blobs removed, stairwell kept open.
Multi-wing sheets	✓ Done	Auto-picks the wing with the most door clusters (not just the biggest block); manual wing switch in the app.
Rooms (walk-inside)	✓ Done	Enclosed free-space detection; marker point via distance transform (safe in L-shaped rooms); ≥28 sqft; hairline door-slit leak fixed.
GLB export	✓ Done	Named meshes (floor/wall/glass/column/furniture), glass panes in windows, normals exported (downloaded GLB works in any viewer).

2.2 API hardening (server)

Item	Status	Details
Timeouts & load	✓ Done	120 s wall-clock timeout → 504; concurrency semaphore; GPU-OOM → clean 503.
Bad input handling	✓ Done	Garbage image bytes → 422; oversized upload → 413; photo upload → friendly 503 beta message.
Boot without torch	✓ Done	Perception imported lazily — the slim Docker image boots with no AI libraries; /health reports mode.
Temp-file race	✓ Done	/scene.glb uses unique temp files with background cleanup.
CORS	✓ Done	ALLOWED_ORIGINS env var, trailing-slash tolerant.

2.3 Viewer (frontend)

Item	Status	Details
------	--------	---------

Landing + theme	✓ Done	Landing page, orange re-theme, viewer HUD and keyboard shortcuts (shipped 13 Jul).
PBR materials	✓ Done	Procedural plaster / floor tile / concrete via box-projected UVs — no texture downloads.
Lighting & sky	✓ Done	Building-scale sun with tuned shadows (verified on a real GPU), sky dome, distance fog, 400 m ground.
Walk inside rooms	✓ Done	Beacon per room; click or keys 1–9 glides the camera to eye level (5.2 ft); gaze aims through the room; camera lantern lights interiors; GSAP lag fix for weak GPUs.
Sample building	✓ Done	■ Try-a-sample 2BHK (40.5×28.6 ft, 4 rooms, 6 doors, 6 windows) works with NO backend — instant demo even if the API is asleep.
Back button	✓ Done	Browser Back/Forward/refresh all behave correctly (history state fix).

2.4 Deploy pack & tests

Item	Status	Details
Docker image	✓ Done	server/Dockerfile — slim, no torch; .dockerignore keeps it small.
Render blueprint	✓ Done	render.yaml — one-click API deploy with health check.
Vercel config	✓ Done	vercel.json — frontend auto-detected.
Deploy guide	✓ Done	docs/DEPLOY.md — full step-by-step (~20 min).
Test suite	✓ Done	110 green: 97 Python (pytest) + 13 frontend (vitest). Includes golden per-plan pins, sample-plan pins, deploy boot test.

3. Accuracy — how it is tested and how accurate it is

Tested on **4 real client sheets** (the corpus in plans/, kept out of git) plus 1 synthetic sample, on **4 parameters**: building envelope size, door snapping, window count, and room detection. Golden tests pin these numbers so any future change that degrades them fails the test suite.

Plan	Envelope	Doors snapped	Windows	Rooms
Sample 2BHK (synthetic)	40.5 × 28.6 ft (exact)	6 / 6 (100%)	6 / 6	4 / 4
FLOOR PLAN (Architrave)	113.9 × 35.6 ft, stable	11 / 12 (92%)	1 — sheet has no window data (dead text, no symbols); needs client info	7
Combined flat numbers	80.0 × 47.8 ft	15 / 20 (75%)	27 tagged	—
Neelachala FIRST FLOOR w0	39 × 97 ft	10 / 28 (36%)	20 tagged	0 — open envelope (missing outer walls on sheet)
Neelachala FIRST FLOOR w1	38.4 × 65.5 ft vs client-confirmed 38.25 × 64.2 (within ±2.5% — the accuracy anchor)	4 / 12 (33%)	(in 20 above)	0 — open envelope

Reading the numbers: envelope size (the headline dimension a builder checks first) is within 2.5% of the client-confirmed figure. Door snapping is strong on clean single-flat sheets (92–100%) and weaker on dense multi-flat sheets. Window detection depends on the sheet having a window layer or live tags.

4. Tasks left to do

4.1 Launch (today)

Item	Status	Details
Push repo to GitHub	✓ Done	Confirmed done by you today.
Deploy API on Render	■ To do	Dashboard → New → Blueprint → pick repo; render.yaml does the rest. Verify /health.
Deploy app on Vercel	■ To do	Import repo; set env var VITE_API_BASE = the Render URL.
Connect CORS	■ To do	On Render set ALLOWED_ORIGINS = the Vercel URL.
Smoke test	■ To do	Sample button, CAD PDF upload, photo beta message, Back button, walk-inside beacons.

4.2 Nice-to-have before you leave

Item	Status	Details
Demo video (MP4)	■ To do	Short screen capture of sample → walk-inside for sharing.
README refresh	■ To do	Root README still describes the old client-side prototype, not the current engine.
Favicon / OG image	■ To do	Browser-tab icon and link-preview card.

4.3 Known improvements for later (v1.1+)

Item	Status	Details
Photo/scan support	■ Partial	Code path exists (CubiCasa perception); needs a GPU host + torch install. Currently answers 'beta'.
Photoreal render button	■ To do	Researched: ControlNet via fal.ai/Replicate API (decision recorded; no self-hosting).
Furniture	■ To do	No furniture in v1 anywhere (sample included) — the sample runs through the real engine, which builds walls/doors/windows/columns only. Furniture-symbol detection is a v1.1+ effort.
Room name labels	■ To do	FIRST FLOOR sheet has live room-name text — could label each beacon.
Neelachala beacons	■ To do	Sheets have open envelopes, so room detection finds 0 rooms; needs envelope-sealing logic.
Door snap on dense sheets	■ To do	Raster gap-finder is fragile at ~5.5 ft openings; endpoint-first snapping is the planned fix.
Architrave windows	■ To do	Blocked on client: sheet has no window data; need counts/locations or a re-export with live text.
Title-text wing pick	■ To do	Pick the wing nearest the 'FLOOR PLAN' title text instead of door count.

5. How to access and run everything

5.1 Run locally (Windows)

Backend: open a terminal in D:\3D React Viewer\server, activate the venv, then `uvicorn main:app --port 8000` (or use `run_backend_gpu.bat`). Restart uvicorn after any server-file change.

Frontend: in D:\3D React Viewer run `npm run dev` → `http://localhost:5173`. Production check: `npm run build` then `npx vite preview --port 4173`.

5.2 Run the tests

Server (97 tests): from server/ with the venv active: `pytest`. Golden corpus tests auto-skip if plans/ is absent.

Frontend (13 tests): from the repo root: `npm test`.

Batch accuracy: `python batch_eval.py` prints the per-plan scorecard.

5.3 Live URLs (fill in after today's deploy)

API: `https://_____.onrender.com` (health check at `/health`) • App: `https://_____.vercel.app`

Note: the free Render instance sleeps; the first upload after idle takes ~30 s. The sample button is instant regardless.

5.4 In the app

Open the app → ■ **Try a sample** for the instant 2BHK, or drop a CAD PDF in the sidebar. Keys 1–9 (or click a beacon) walk inside each room at eye level; the HUD lists shortcuts. Multi-wing sheets: switch wings in the sidebar (Neelachala envelope flat = Wing 1). Download the `.glb` from the viewer — it opens correctly in any standard 3D viewer.

5.5 Key files & docs

docs/DEPLOY.md — deploy walkthrough • render.yaml / vercel.json — hosting configs • server/pdf_vector.py — parsing engine • server/main.py — API • src/ — React viewer • docs/CUBICASA-INTEGRATION.md — photo-beta architecture decision.

Prepared by Claude for the Drishti v1 launch — 18 July 2026. 110/110 tests green at time of writing.